

Análisis y Reflexión Taller 1 IA

Integrantes del equipo:

- Carolina Ceballos Gómez - 202510267
- Juan Felipe Leon - 202510397
- Miguel Aguilar - 202411646
- Andrés Sanabria - 202411507

a) ANÁLISIS

Punto 1: Respuesta a Baliza de Emergencia (depthFirstSearch)

Complejidad en espacio: $O(b \cdot m)$, siendo b =el factor de ramificación, es decir el número de sucesores de un nodo, y m =la profundidad máxima del árbol de búsqueda.

Complejidad en tiempo: $O(b^m)$, siendo b =el factor de ramificación y m =la profundidad máxima.

¿El algoritmo es completo? Si es completo, ya que sabemos que el mapa es finito, que el robot sólo puede moverse en 4 direcciones y que el número de estados es limitado. Además, con nuestra implementación, donde guardamos los nodos/estados ya visitados, estamos previniendo los ciclos o recorridos infinitos.

¿El algoritmo encuentra la solución óptima siempre o en cuáles condiciones? El algoritmo no encuentra la solución óptima, ya que DFS siempre retorna la primera solución que encuentra, sin verificar si existen otras soluciones más óptimas o eficientes en términos de costo o distancia. Solo cuando la primera solución encontrada es casualmente la más óptima.

Es un algoritmo que puede encontrar la solución rápido y que no utiliza mucha memoria en comparación con otros algoritmos como BFS. Pero también puede perder bastante tiempo en ramas equivocadas y tampoco garantiza encontrar la solución óptima, algo que sí hace BFS (con costos uniformes) o A^* . Además, desde el punto de vista cuantitativo, DFS, aunque expande menos nodos que BFS o A^* , las soluciones que devuelve son más costosas. Respecto al tiempo, este depende de los nodos expandidos y de la profundidad a la que se deba llegar, así que si depth-FirstSearch explora ramas equivocadas, el tiempo también puede aumentar significativamente.

Punto 2: Camino Más Corto a un Sobreviviente (breadthFirstSearch)

Complejidad en espacio: $O(b^d)$, siendo b =el factor de ramificación y d =la profundidad de la solución retornada.

Complejidad en tiempo: $O(b^d)$, siendo b =el factor de ramificación y d =la profundidad de la solución retornada.

¿El algoritmo es completo? Si es completo, ya que sabemos que el mapa es finito, que el robot solo puede moverse en 4 direcciones y que el número de estados es limitado. Además, con

nuestra implementación, donde guardamos los nodos/estados ya visitados, estamos previniendo los ciclos o recorridos infinitos.

¿El algoritmo encuentra la solución óptima siempre o en cuáles condiciones? No siempre encuentra la solución óptima, sólo lo hace cuando los costos son uniformes, pero en el caso del taller, tenemos obstáculos o terrenos con más costos como el agua o escombros, en donde el costo por pasos es mayor que en el piso normal.

BFS explora los nodos/estados por niveles, de esa manera puede encontrar el camino con menos pasos, pero esto implica consumir mucha más memoria pues debe guardar los nodos de cada nivel y después avanzar al siguiente. Por el lado cuantitativo vemos que encuentra soluciones con menor costo comparado con DFS pero expande o igual cantidad o mayor cantidad de nodos que DFS, en cuanto al tiempo, BFS suele tardar más que DFS pues debe explorar el espacio revisando todos los nodos de cada nivel antes de seguir un camino, esto aumenta también el número de expansiones y el tiempo de ejecución.

Punto 3: Navegación en Terreno Peligroso (uniformCostSearch)

Complejidad en espacio: $O(b^{(1+\lfloor C^*/\epsilon \rfloor)})$, siendo b =el factor de ramificación, C^* =costo de la solución óptima y ϵ = el costo mínimo de la acción/paso.

Complejidad en tiempo: $O(b^{(1+\lfloor C^*/\epsilon \rfloor)})$, siendo b =el factor de ramificación, C^* =costo de la solución óptima y ϵ = el costo mínimo de la acción/paso.

¿El algoritmo es completo? Sí, ya que encuentra siempre la solución, pues expande el nodo con el menor costo acumulado (en este contexto, los costos son siempre positivos y pueden variar) y, si la solución existe (en este caso, encontrar/rescatar al sobreviviente), siempre llegará a ella.

¿El algoritmo encuentra la solución óptima siempre o en cuáles condiciones? Si llega a la solución óptima siempre, ya que siempre expande al de menor costo acumulado y esto después de muchas iteraciones lo hará llegar a la solución de costo más bajo, sin embargo, se debe tener cuidado con los casos en donde el costo sea negativo (no aplica para el taller), pues esto compromete la optimidad.

UCS garantiza la solución con el menor costo posible, aunque esto puede comprometer los nodos expandidos (normalmente es una cantidad mayor a la expansión de DFS por ejemplo). Y en cuanto al tiempo de ejecución, uniform-Costsearch puede requerir más tiempo que otros algoritmos, pues utiliza una cola de prioridad y debe estar evaluando los costos acumulados cada vez que explora los nodos de un nuevo nivel.

Punto 4: Búsqueda Informada Eficiente (aStarSearch con las heurísticas de distancia Manhattan y distancia Euclidiana)

b = branching factor

d = profundidad de la solución óptima

Complejidad en espacio: $O(b^d)$, en el peor caso, debe recorrer todas las ramas haciendo un recorrido similar a BFS. En caso de que la heurística sea poco informativa su tiempo incrementa exponencialmente. Pero si la heurística es “perfecta”, solo se recorre el camino óptimo en tiempo lineal.

Complejidad en tiempo: $O(b^d)$ ya que debe almacenar todos los nodos hasta la profundidad d . En caso que la heurística sea poco informativa se guarda una cantidad de nodos que crece exponencialmente

¿El algoritmo es completo? El algoritmo es completo en nuestro caso pues existe una cantidad finita de nodos y los costos siempre son >0 . También depende de si la heurística es admisible, o sea, que nunca debe sobreestimar el costo del trayecto.

¿El algoritmo encuentra la solución óptima siempre o en cuáles condiciones? En el taller, el algoritmo sí encuentra la solución siempre y cuando la heurística que se utilice sea admisible, en la pregunta de abajo responderemos los 2 casos que tenemos en este punto. (Además de que ya sabemos que los costos son positivos y el factor de ramificación es finito).

¿Las heurísticas son consistentes? La heurística euclidiana hace que la distancia directa desde un nodo hasta la meta nunca sea mayor que ir primero a un vecino y luego a la meta, o sea nunca sobreestima y es admisible. Si el costo real entre nodos es al menos su distancia geométrica, entonces se cumple $h(n) \leq c(n, n') + h(n')$. Por eso también es consistente.

La Heurística Manhattan es consistente pues en un plano donde solo se permiten movimientos horizontales y verticales con costos positivos, la reducción en la heurística al moverse a un vecino es como máximo el costo del movimiento. Entonces se cumple $h(n) \leq c(n, n') + h(n')$ y también la vuelve admisible.

Punto 5: Rescate de Múltiples Sobrevivientes (aStarSearch con survivorHeuristic)

Complejidad en espacio: $O(b^d)$ ya que A^* debe almacenar en memoria todos los nodos generados. En el peor caso, la cantidad de nodos almacenados crece exponencialmente con la profundidad de la solución.

Complejidad en tiempo: En el peor caso es $O(b^d \cdot v^2)$, ya que A^* puede expandir hasta b^d nodos. Además, por cada nodo generado se calcula la heurística utilizando el algoritmo de Prim para hallar el MST, usando una matriz de adyacencia, lo cual tiene un costo $O(v^2)$ (considerando que se revisan todas las posibles conexiones entre los vértices). Por lo tanto, el costo total resulta de multiplicar el número de nodos expandidos por el costo de evaluar la heurística en cada uno.

¿El algoritmo es completo?

El algoritmo es completo, dado que el espacio de estados es finito, el factor de ramificación es finito y los costos de las acciones son estrictamente mayores que 0. Bajo estas condiciones, A^* no puede entrar en ciclos infinitos con costo decreciente y eventualmente expandirá todos los nodos necesarios hasta encontrar la solución.

¿El algoritmo encuentra la solución óptima siempre o en cuáles condiciones?

Sí, A* encuentra la solución óptima siempre que la heurística sea admisible. En este caso, la heurística basada en el MST, calcula el costo mínimo para conectar todos los sobrevivientes ignorando obstáculos y restricciones del entorno, lo que garantiza admisibilidad al no superar el costo.

¿Las heurísticas son consistentes?

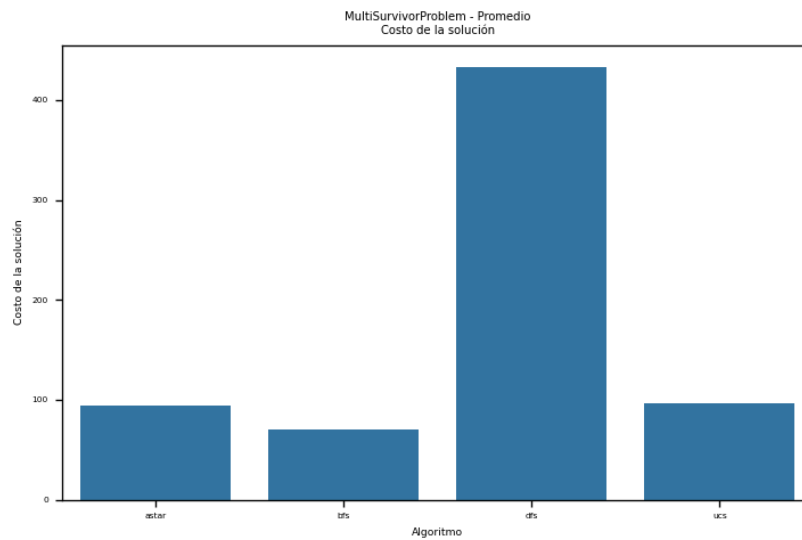
Sí, dado que el valor estimado desde un nodo nunca es mayor que el costo de ir a un vecino más la estimación desde ese vecino. Adicionalmente, el MST construido con distancias Manhattan también respeta esta propiedad, ya que al movernos a un nodo vecino el valor de la heurística no puede disminuir en una cantidad mayor que el costo del movimiento realizado, lo que implica que la heurística sea consistente.

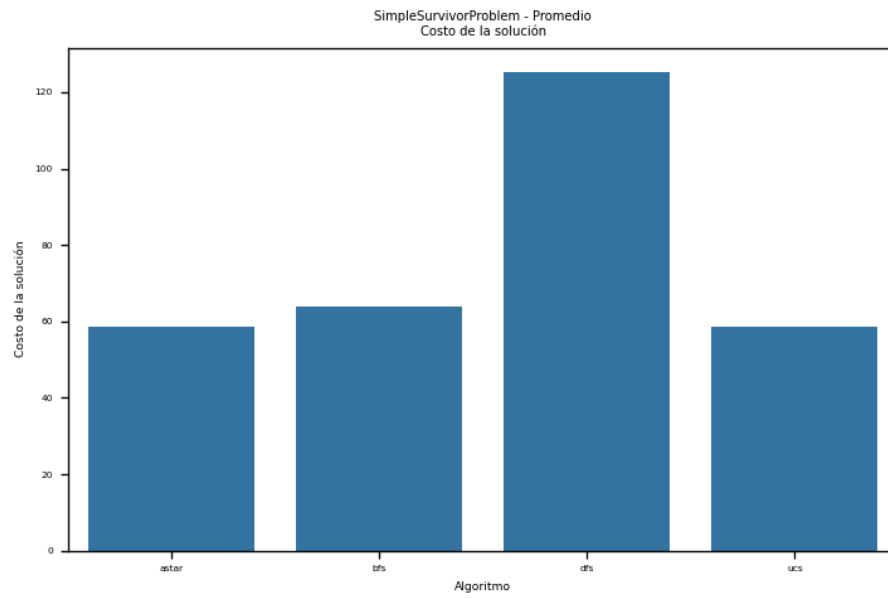
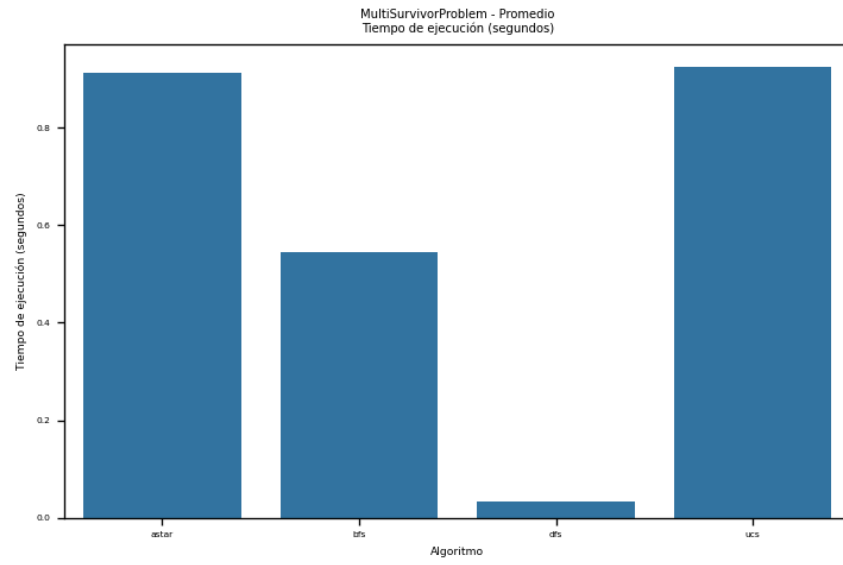
Tabla Comparativa entre Algoritmos

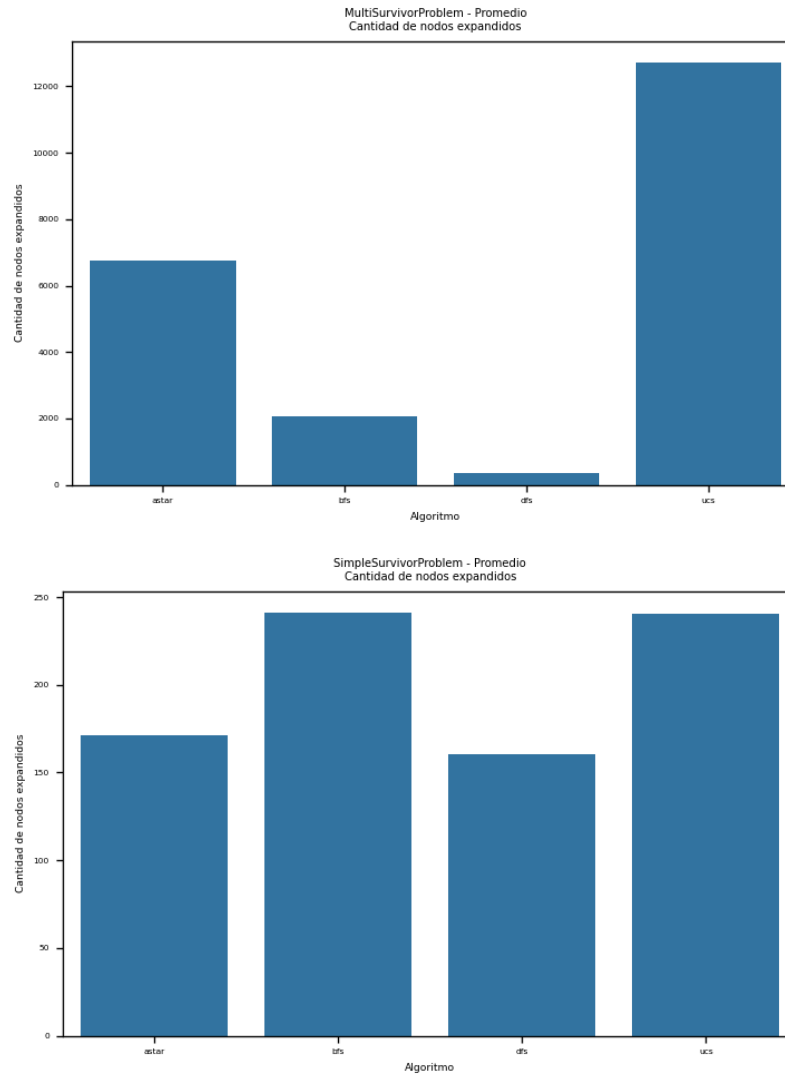
Layout	Algoritmo	Costo Total	Nodos Expandidos	Tiempo (en s)
Burning Office (Simple)	DFS	132	96	0,0
	BFS	164	306	0,0
	UCS	128	309	0,0
	A*	128	309	0,0
bigCollapsed Building (Simple)	DFS	17	15	0,0
	BFS	17	69	0,0
	UCS	17	70	0,0
	A*	17	70	0,0
Bigcollapsed Building (Multi)	DFS	606	291	0,1
	BFS	BUCLE		
	UCS	77	105032	42,7
	A* euclidian	77	90397	54,3
	A* manhattan	77	88534	53,7

	A* survivor	79	20275	12,4
DowntownComplex (Multi)	DFS	1963	1188	0,6
	BFS	BUCLE		
	UCS	273	210898	175.7
	A* euclidian	273	196044	269.4
	A* manhattan	273	193630	268.1
	A* survivor	273	98272	142.9

Adicional a estas tablas comparativas, optamos por realizar un análisis mediante gráficas del comportamiento de cada algoritmo con base en las métricas mencionadas previamente, en cada uno de los layouts propuestos.







En cuanto al costo de la solución, DFS fue el que presentó los resultados menos favorables en los mapas más complejos, principalmente porque no garantiza optimalidad y puede encontrar caminos mucho más largos de lo necesario. Por otro lado, BFS, UCS y A* lograron producir soluciones óptimas siempre que el algoritmo alcanzaba a terminar su ejecución correctamente.

Si observamos la cantidad de nodos expandidos, es evidente que BFS y UCS son los algoritmos que más exploran, o sea, tienen un mayor consumo de recursos. A* logra mejorar este comportamiento cuando utiliza heurísticas básicas como Manhattan o Euclidiana pero la diferencia más significativa se da cuando se usa la survivorHeuristic donde se observa una disminución considerable en el número de nodos expandidos. Por su parte, DFS expande pocos nodos, pero esto ocurre a costa de sacrificar la calidad de la solución.

En relación con el tiempo de ejecución, BFS y UCS no son tan favorables. Por otro lado, A* con una heurística adecuada un balance entre tiempo y calidad de solución. DFS, aunque es rápido y constante en tiempo, no ofrece garantías de solución óptima.

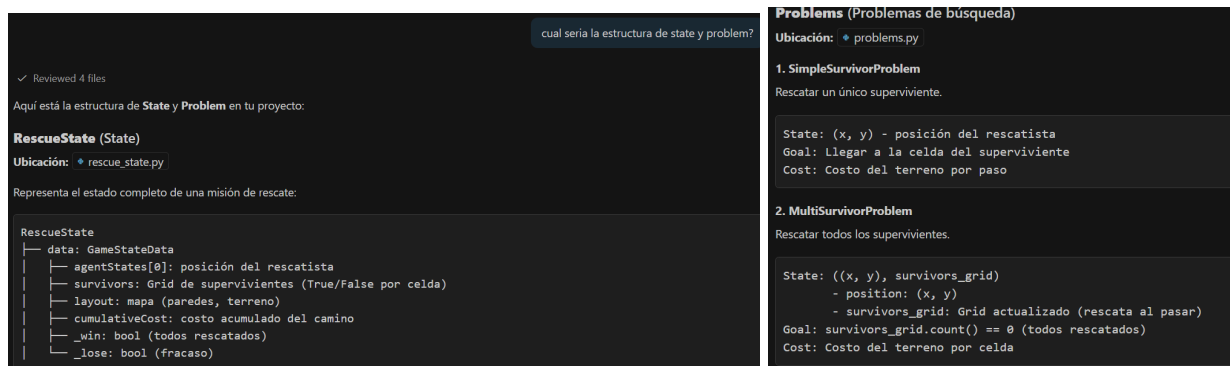
b) REFLEXIÓN

Como equipo comenzamos el desarrollo de este taller de forma autónoma sin ayudas externas, cuando nos encontrábamos con algún problema o bloqueo acudimos primero entre nosotros para recibir alguna retroalimentación u orientación, sin embargo, cuando las dudas sobrepasaban el conocimiento de los 3 o después de intentarlo varias veces de diferentes maneras y no llegábamos a la solución, sí empezamos a utilizar un poco de apoyo de la IA. Creemos que este apoyo fue importante no solo para lograr el correcto desarrollo del taller, sino también para aclarar algunas dudas conceptuales de lo visto en clase. Los aprendizajes que obtuvimos gracias a las correcciones de la IA fueron algunas mejoras secundarias para “simplificar” un poco el código o tal vez no redundar tanto. Lo valioso de la IA es que no nos arrojaba solo una respuesta sino que nos mostraba varias opciones dándonos la posibilidad de escoger cuál se asemejaba más a nuestro estilo y también nos explicaba cada opción y las diferencias entre ellas. Esto nos ayudó bastante a comprender lo que la IA estaba haciendo y por eso sabemos que podríamos explicar la solución dada en una sustentación.

Otra funcionalidad de la IA que utilizamos, fue mandarle los errores que nos sacaba Visual Studio Code y que no comprendemos, lo clave aquí fue que no le pedíamos que nos corrigiera el código y ya si no que nos diera una lista de posibles fallas de esa manera nos dabamos cuenta de los errores pero nosotros mismos los corregimos. Respecto a la construcción del prompt, creemos que fue fácil hacer uno adecuado donde primero indicaremos cómo quería que nos respondiera (si de una manera técnica o más sencilla), la inquietud o falla que teníamos, y le decíamos que sin darnos la respuesta nos guiara para llegar a ella nosotros mismos, o en algunos casos que nos diera algunas alternativas para mejorar el código.

Y finalmente mostraremos los prompts utilizados, y las respuestas dadas por la IA:

Usando el Copilot de Visual Studio Code:



The screenshot displays the Visual Studio Code interface with the Copilot extension. On the left, a chat window shows a prompt asking for the structure of state and problem, and the AI's response defining the `RescueState` and `Problems` classes. The `RescueState` class is defined with attributes like `agentStates`, `survivors`, `layout`, `cumulativeCost`, `_win`, and `_lose`. The `Problems` class contains two methods: `SimpleSurvivorProblem` and `MultiSurvivorProblem`, each with its own state, goal, and cost function.

```
RescueState (State)
Ubicación: rescue_state.py

Representa el estado completo de una misión de rescate:

RescueState
├── data: GameStateData
│   ├── agentStates[0]: posición del rescatista
│   ├── survivors: Grid de supervivientes (True/False por celda)
│   ├── layout: mapa (paredes, terreno)
│   ├── cumulativeCost: costo acumulado del camino
│   ├── _win: bool (todos rescatados)
│   └── _lose: bool (fracaso)
```

```
Problems (Problemas de búsqueda)
Ubicación: problems.py

1. SimpleSurvivorProblem
Rescatar un único superviviente.

State: (x, y) - posición del rescatista
Goal: Llegar a la celda del superviviente
Cost: Costo del terreno por paso

2. MultiSurvivorProblem
Rescatar todos los supervivientes.

State: ((x, y), survivors_grid)
    - position: (x, y)
    - survivors_grid: Grid actualizado (rescata al pasar)
Goal: survivors_grid.count() == 0 (todos rescatados)
Cost: Costo del terreno por celda
```


ayúdame a resolverlo sin que escribas código, solo dime que partes fallan y cual lógica deberían tener

✓ Considered failures and logic improvements for uniformCostSearch

Aquí están los problemas lógicos:

1. `vertex` **nunca cambia**

Lo que hace mal: El loop usa `while not problem.isGoalState(vertex):` pero `vertex` siempre apunta al estado inicial.

Lógica que debería tener: Después de procesar un nodo, `vertex` debe actualizarse al siguiente nodo que saques del frontier. Sin eso, el loop procesa siempre los mismos sucesores del inicio.

2. El frontier recibe solo `stepCost`, no el costo acumulado

Lo que hace mal: `frontier.push(sucesor, sucesor[2])` solo usa el costo del paso actual (1, 2, 3, o 5).

Lógica que debería tener: La prioridad del frontier debe ser el costo total desde el inicio hasta ese sucesor. Necesitas acumular: `costo_anterior + stepCost`. Esto significa rastrear en un diccionario aparte cuál es el costo acumulado de cada nodo.

6. No hay manejo de frontier vacío

Lo que hace mal: El loop `while not frontier.isEmpty():` podría ejecutarse indefinidamente sin control.

Lógica que debería tener: Si el frontier se vacía sin encontrar el goal, debes retornar una lista vacía (sin solución).

Resumen de flujo correcto:

1. Inicializa: frontier con estado inicial y costo 0

2. Loop mientras frontier no esté vacío:

Pop el nodo de menor costo

Si es goal → retorna camino

Si ya fue visitado → skip

Marca como visitado

Para cada sucesor:

Calcula nuevo costo acumulado

Si es peor que lo conocido → ignora

Si es mejor → actualiza costo y acciones, re-agrega al frontier

3. Si frontier vacío: retorna []

3. No se evitan duplicados en el frontier

Lo que hace mal: Si un nodo ya fue visitado, lo vuelves a agregar al frontier gastando recursos.

Lógica que debería tener: Antes de agregar un sucesor al frontier, verifica si ya está en `visited`. Solo agrega si es nuevo.

4. El diccionario `actions` almacena el costo, no el camino

Lo que hace mal: `actions=[problem.getStartState()]` guarda un número, y después `actions[next]=next[1]` guarda solo una acción individual, no una lista de acciones.

Lógica que debería tener: `actions` debe guardar la secuencia completa de acciones desde el inicio hasta ese nodo (como lista). Cuando descubres un nuevo nodo, haces `actions[nuevo] = actions[padre] + [accion]`.

5. Falta manejar nodos descubiertos múltiples veces

Lo que hace mal: Si encuentras un camino más barato a un nodo ya visitado, no lo re-expands con el nuevo costo mejor.

Lógica que debería tener: Necesitas un diccionario `costs` que guarde el mejor costo encontrado hasta cada nodo. Si encuentras un camino más barato (`newCost < costoAnterior`), actualiza el costo, las acciones, y re-agrega el nodo al frontier con la nueva prioridad.

Usando Chat-GPT 5:

Obtener Plus

que error es? File "C:\Users\Usuario\Documents\Universidad\Semestre 3\DPO\InteligenciaArtificial-202610\main.py", line 218, in <module> runMission(**args)

File "C:\Users\Usuario\Documents\Universidad\Semestre 3\DPO\InteligenciaArtificial-202610\main.py", line 199, in runMission episode.run()

File "C:\Users\Usuario\Documents\Universidad\Semestre 3\DPO\InteligenciaArtificial-202610\world\game.py", line 577, in run agent.registerInitialState(self.state.deepCopy())

File "C:\Users\Usuario\Documents\Universidad\Semestre 3\DPO\InteligenciaArtificial-202610\algorithms\agents.py", line 66, in registerInitialState self.actions = self.searchFunction(problem) # Find path using search algorithm

File "C:\Users\Usuario\Documents\Universidad\Semestre 3\DPO\InteligenciaArtificial-202610\algorithms\search.py", line 93, in uniformCostSearch while not utils.isEmpty(heap):

AttributeError: module 'algorithms.utils' has no attribute 'isEmpty'

¿Qué significa el error?

El mensaje dice:

Código

AttributeError: module 'algorithms.utils' has no attribute 'isEmpty'

Y señala esta línea:

Python

while not utils.isEmpty(heap):

Eso significa que estás intentando usar una función:

Python

utils.isEmpty()

✗ Pero esa función NO existe en `utils`.